# Lecture #6: Graphics for Numeric Variables

## STAT 200 – Introductory Statistics

Torin Quinlivan

Knox College

October 28, 2026

# Today's Objective

After today's lecture, you should:

- create the following univariate graphics for numeric variables:
  - histogram
  - density plot
  - overlay plot
  - box-and-whiskers plot
- create the following bivariate graphics for numeric variables:
  - side-by-side boxplot
  - line plot
  - scatter plot
- modify graphics to make them presentation-worthy

# Reading in the Data

To read into R the data we will use for this lecture, run the following code:

```
dt = read.csv("http://rfs.kvasaheim.com/data/crime.csv")
```

This will read the dataset in from Dr. Forsberg's website.

If the `.csv` file is on your computer, use `read.csv("file path")`, i.e.

```
dt = read.csv("/Users/tquinlivan/R/crime.csv")
```

`read.csv` reads the `.csv` file into `R` as a data frame.

If you want the columns attached as individual vectors, use the following function:

```
attach(dt)
```

# Three Graphics Packages

Once again, there are three main packages to use while plotting in R:

- `base` (This is the built-in package in R.)
- `grid`
- `ggplot2`

They all use a different way of thinking about how graphics should be created. `grid` and `ggplot2` extend base R (`grid` ships with R; `ggplot2` must be installed) and can make prettier plots.

# How to Think About Graphics

We will discuss the base R functions in this lecture.
How should we think about our process for creating plots?

1. Plan your "painting".
2. Define the parameters of the "canvas".
3. Start the "canvas".
4. Add to the "canvas".

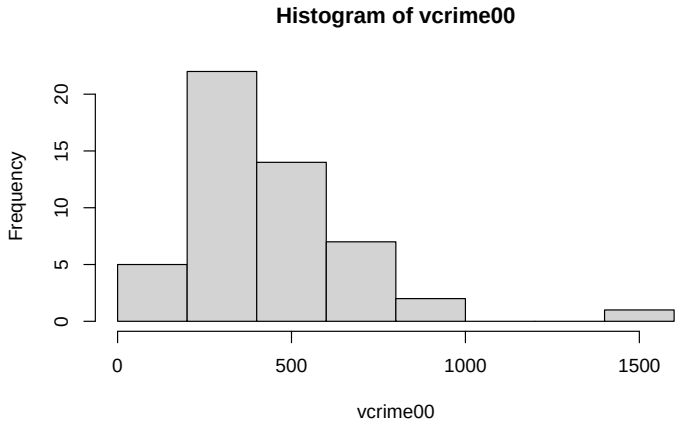
# Univariate Graphics for Numeric Variables

The **univariate** graphics we will discuss are:

- histogram
- density plot
- overlay plot
- box-and-whiskers plot

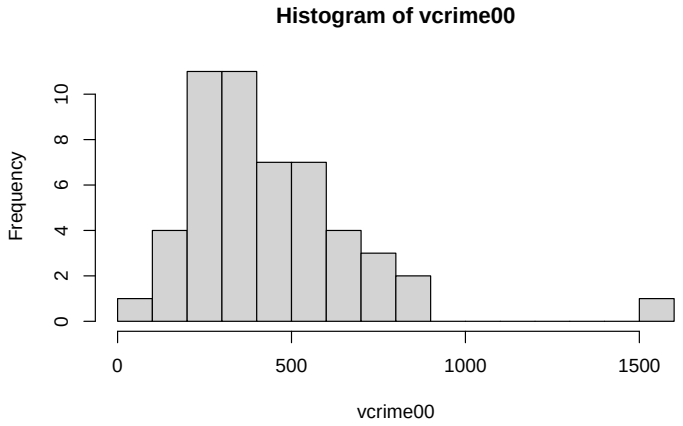
# Histogram

```
hist(vcrime00)
```



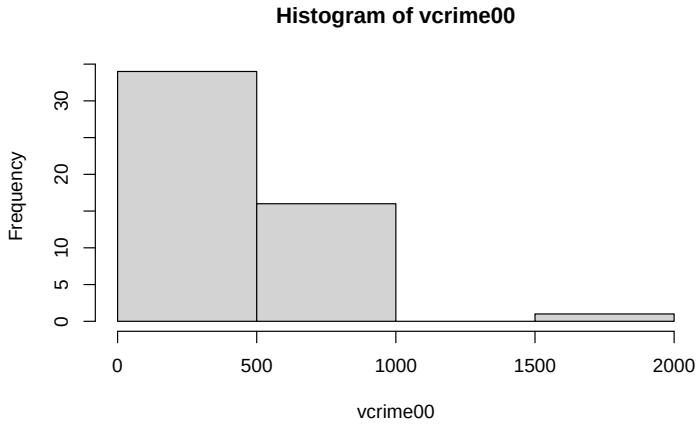
# Histogram

```
hist(vcrime00, breaks=11)
```



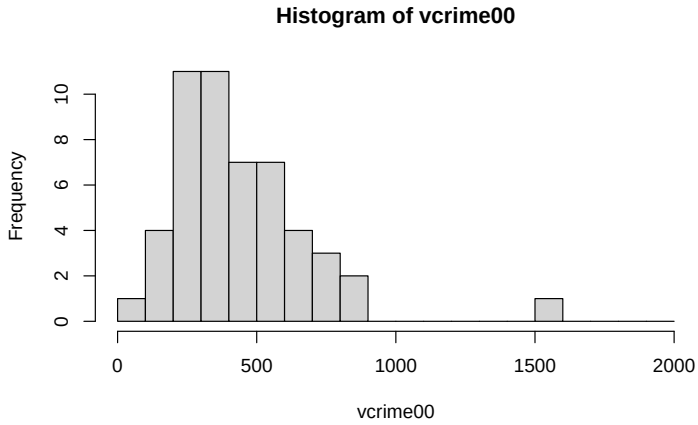
# Histogram

```
hist(vcrime00, breaks=5)
```



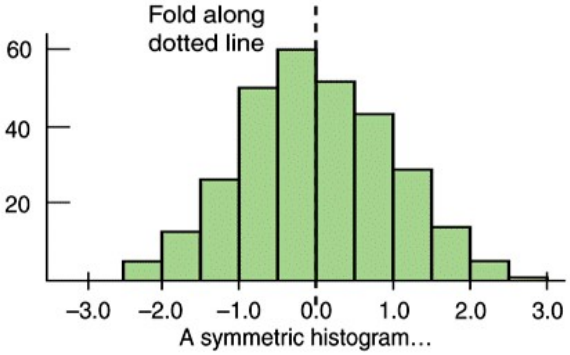
# Histogram

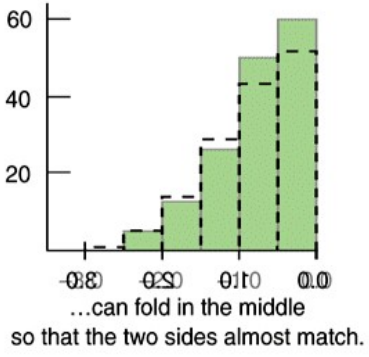
```
hist(vcrime00, breaks=seq(0,2000,100))
```



# Shape of a Histogram

A **symmetric** distribution is one which looks the same on both sides if split in half. This is approximate; no real life distributions will ever be perfectly symmetric.

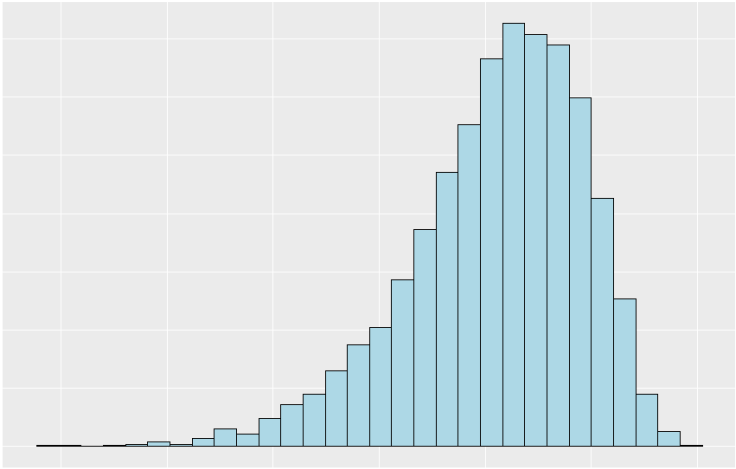


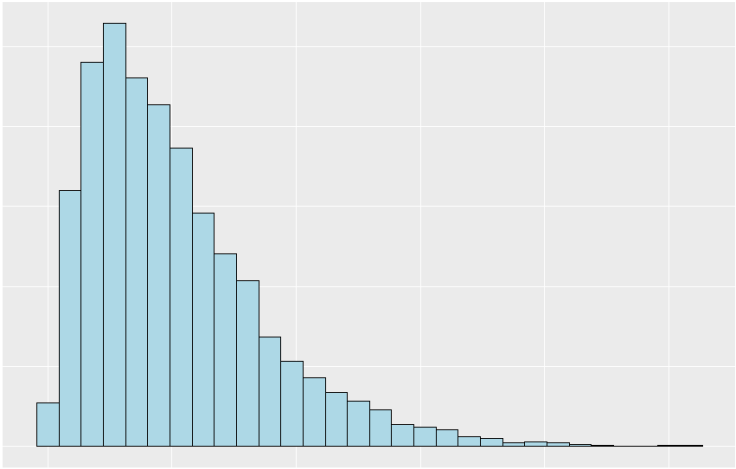
# Shape of a Histogram

If a distribution is not symmetric, we call that distribution **skewed**.
Skewed distributions have long "tails" extending out in one direction.
If the tail is extending out to the left, we say the distribution is **left (or negatively) skewed**.
If the tail is extending out to the right, we say the distribution is **right (or positively) skewed**.

# Left Skewed Example

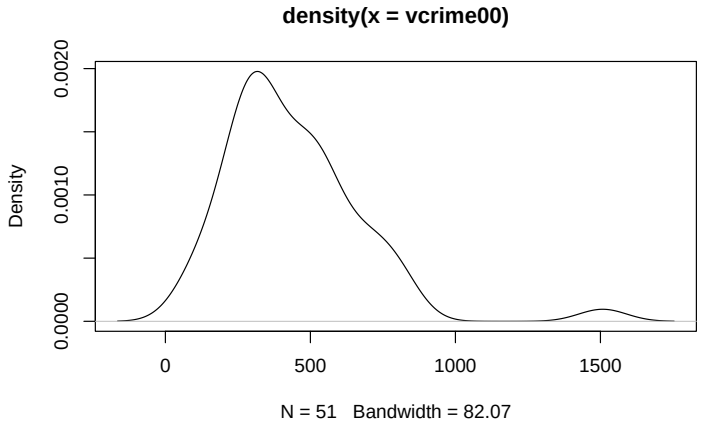


# Right Skewed Example



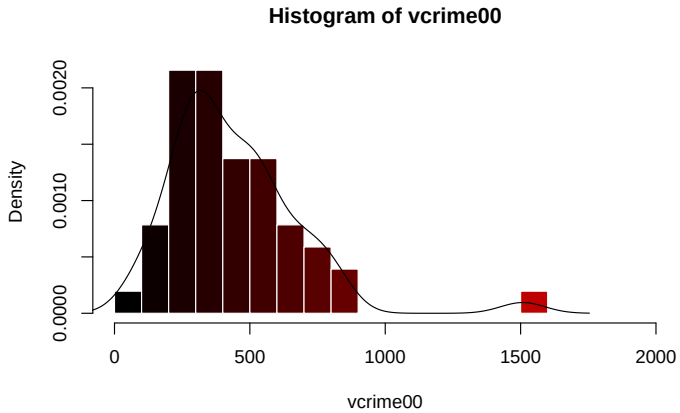
# Density Plot

```
plot(density(vcrime00))
```



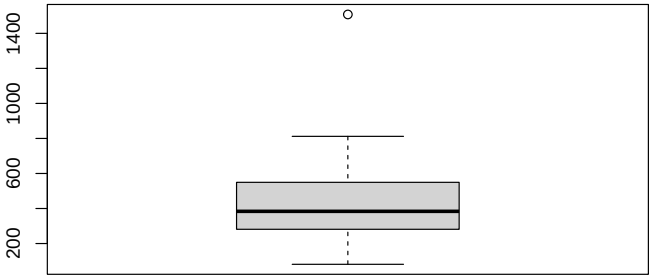
# Density Plot

```
hist(vcrime00, freq=FALSE, breaks=seq(0,2000,100), col=rgb(0:20/20,0,0), border="white")
lines(density(vcrime00))
```



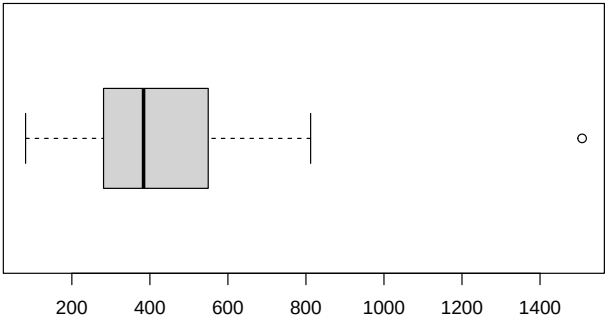
# Boxplot

```
boxplot(vcrime00)
```



# Boxplot

```
boxplot(vcrime00, horizontal=TRUE)
```



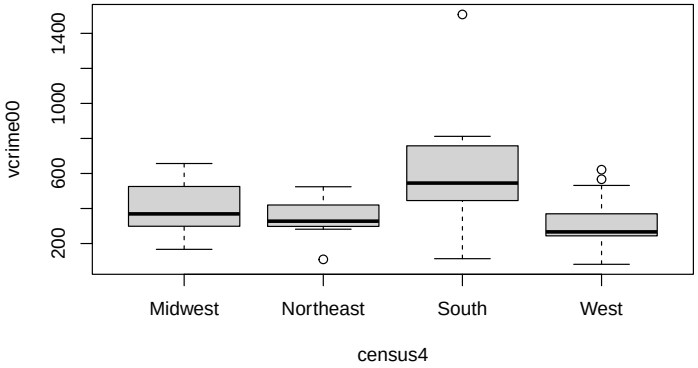
# Bivariate Graphics for Numeric Variables

The **bivariate** graphics we will discuss are:

- side-by-side boxplot
- scatter plot
- line plot

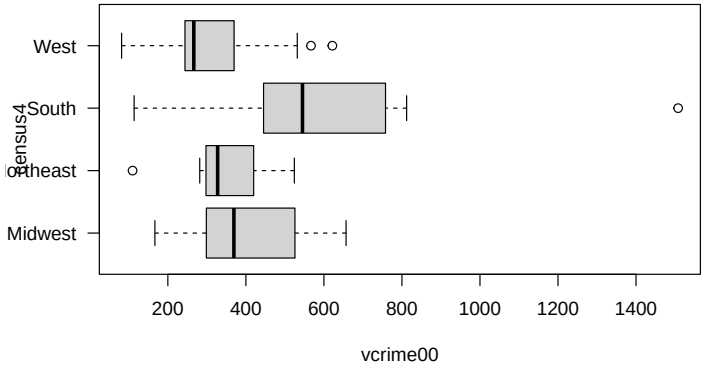
# Side-By-Side Boxplot

```
boxplot(vcrime00 ~ census4)
```



# Side-By-Side Boxplot

```
boxplot(vcrime00 ~ census4, horizontal=TRUE, las=1)
```



# Scatter Plot

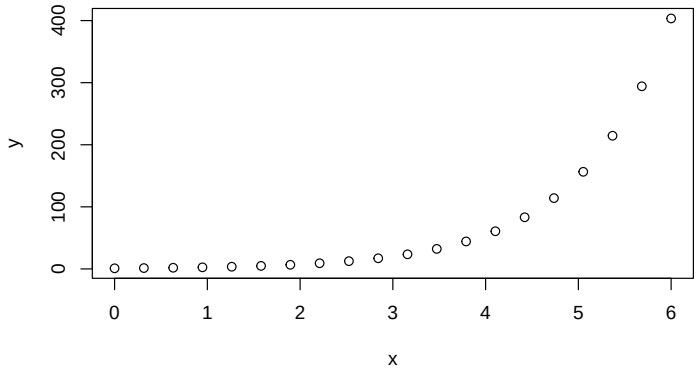
We'll start by making a couple of vectors to plot:

```
x <- seq(0, 6, length.out=20)   # (1)
y <- exp(x)                     # (2)
```

1. Generate 20 evenly spaced numbers between 0 and 6
2. Create a new vector of $e^x$ for each $x$

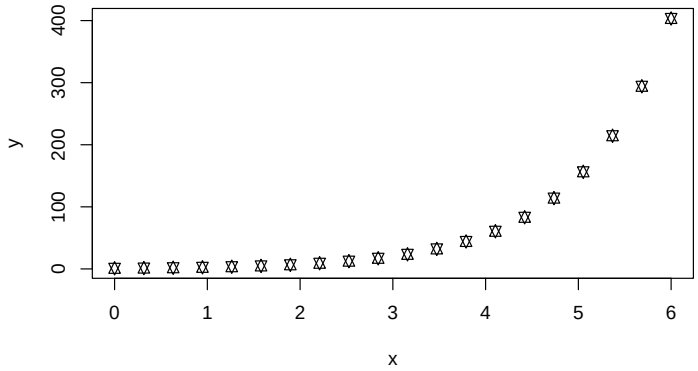
# Plots in R

```
plot(x,y)
```



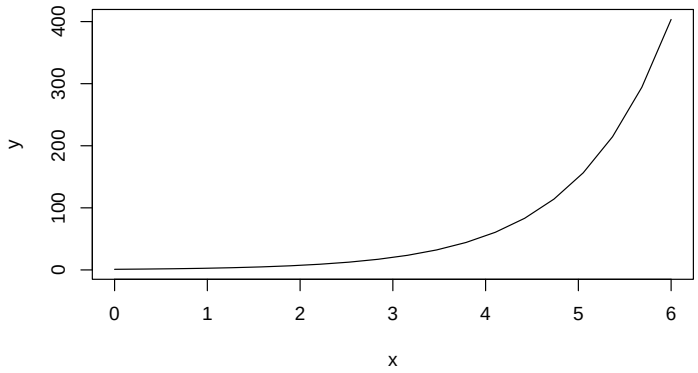
# Plots in R

```
plot(x,y, pch=11)
```



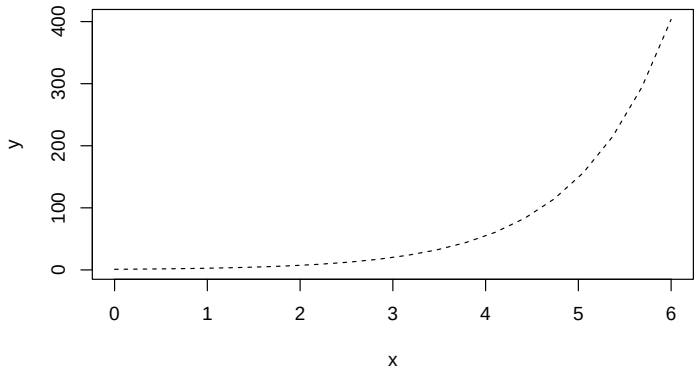
# Plots in R

```
plot(x,y, type="l")
```



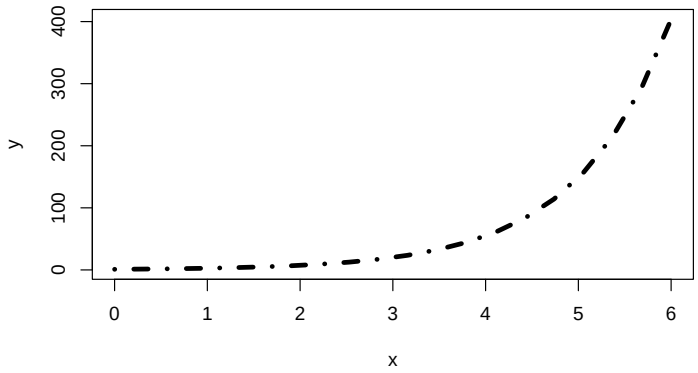
# Plots in R

```
plot(x,y, type="l", lty="dashed")
```



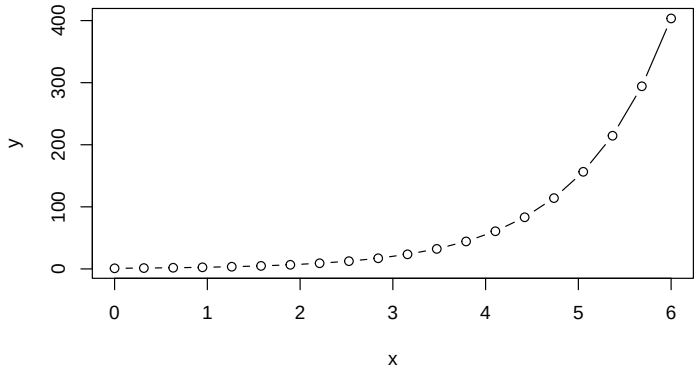
# Plots in R

```
plot(x,y, type="l", lty="dotdash", lwd=4)
```



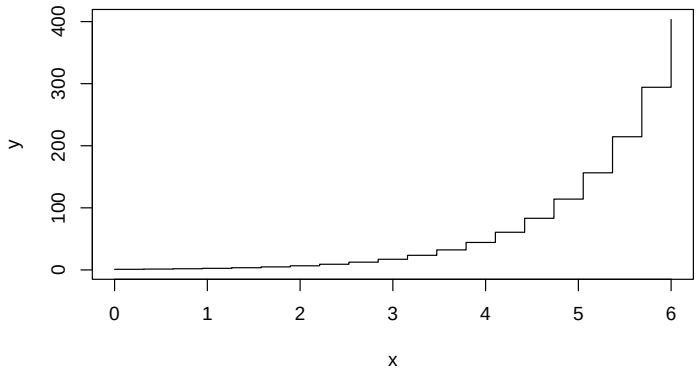
# Plots in R

```
plot(x,y, type="b")
```

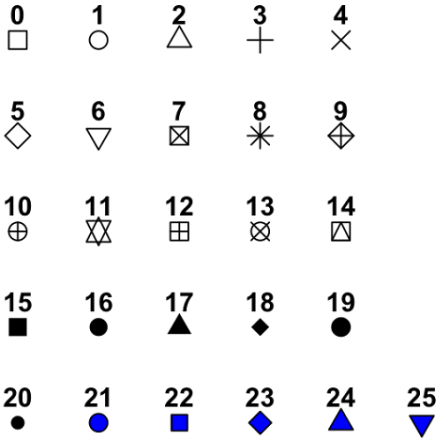


# Plots in R

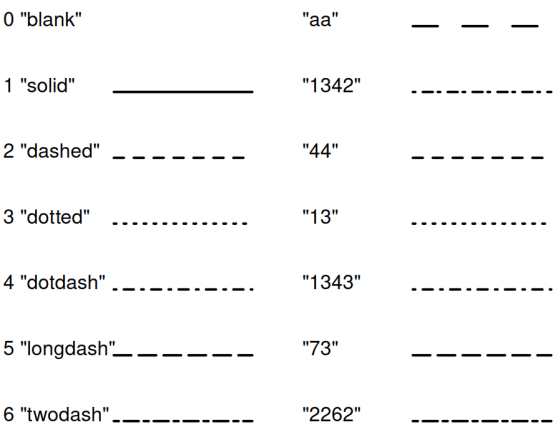
```
plot(x,y, type="s")
```



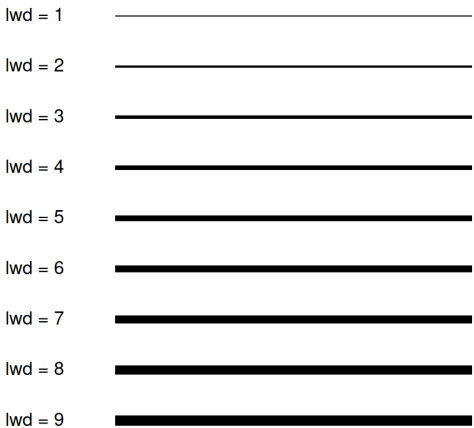
# R Point Types (`pch =`)



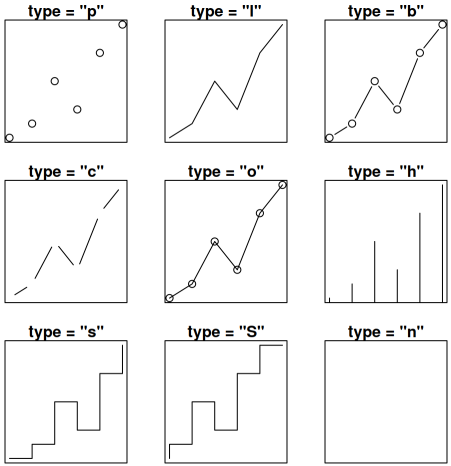
# R Line Types (`lty =`)



# R Line Widths (`lwd =`)

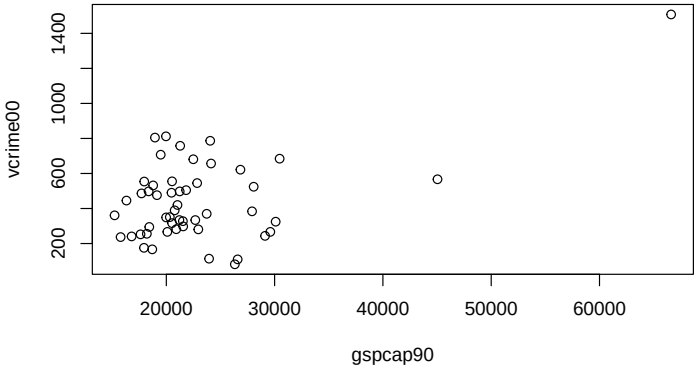


# R Plot Types (`type =`)



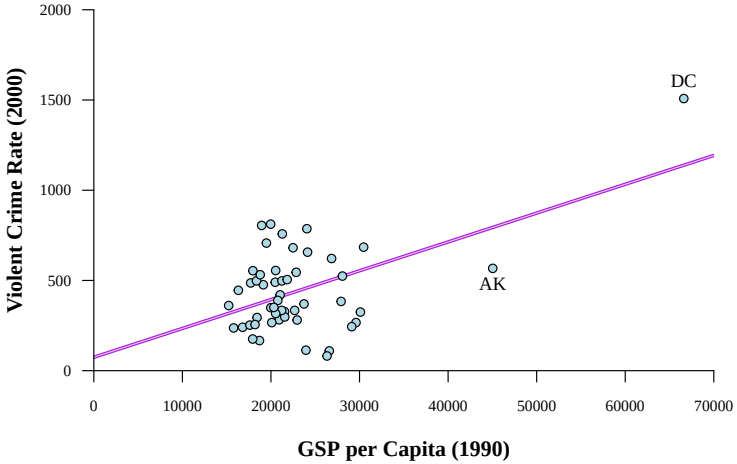
# Scatter Plot

```
plot(vcrime00 ~ gspcap90)
```



# How Do We Make This Plot?

Suppose we want to make a plot like this. How can we make something this fancy?



# The Parameters

```
par(mar=c(4,4,1,1))                # (1)
par(family="serif", font.lab=2)    # (2)
par(cex.lab=1.2, cex.axis=0.8)     # (3)
par(xaxs="i", yaxs="i")            # (4)
par(las=1)                         # (5)
```

1. sets the margins to be 4, 4, 1, and 1 (lines at the bottom, left, top, and right);
2. specifies the font family is serif, with labels bolded;
3. specifies the labels are 20% larger and values are 20% smaller than default;
4. requires the axes to meet at 0 (no axis padding); and
5. forces all labels to be horizontal (for the sake of your readers).

# The Start

```
plot.new()                                          # (1)
plot.window(xlim=c(0,70000), ylim=c(0,2000))        # (2)
```

1. starts the graphic; and
2. sets the viewing window to $(0, 70000) \times (0, 2000)$.

Note that this will start a generic window of that size/dimension.
Again, some graphing functions have their own "starting" functions. So, in those cases, the above two lines are not needed.

# The Annotation

```
abline(lm(vcrime00~gspcap90), lwd=3, col="purple")          # (1)
abline(lm(vcrime00~gspcap90), lwd=1, col="pink")            # (1)

points(gspcap90 ,vcrime00 , pch=21, bg="lightblue")         # (2)

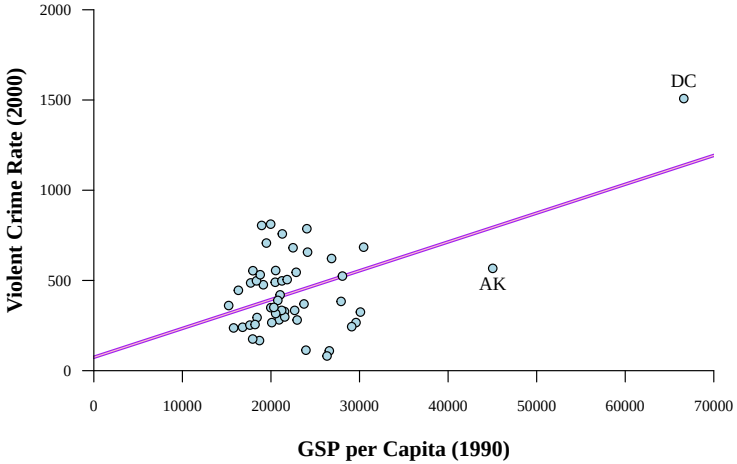
axis (1); axis (2)                                          # (3)
title(xlab="GSP per Capita (1990)")                         # (3)
title(ylab="Violent Crime Rate (2000)")                     # (3)

text(gspcap90[24],vcrime00[24], label=scode[24], pos=3)     # (4)
text(gspcap90[50],vcrime00[50], label=scode[50], pos=1)     # (4)
```

1. adds the line of best fit (looking all sexy);
2. adds the points (light-blue-filled circles);
3. adds the values and labels; and
4. identifies the two outliers.

# Our Plot

All this comes together to produce:



# Maps in R

Note that maps are especially difficult to make, but well worth the effort for geographical data.
Properly making maps is an art form.
There are a lot of features of maps that need to be considered when plotting.
Because of this, maps are especially difficult in any program, even in R.

# Maps in R

There are several packages that help in map creation.

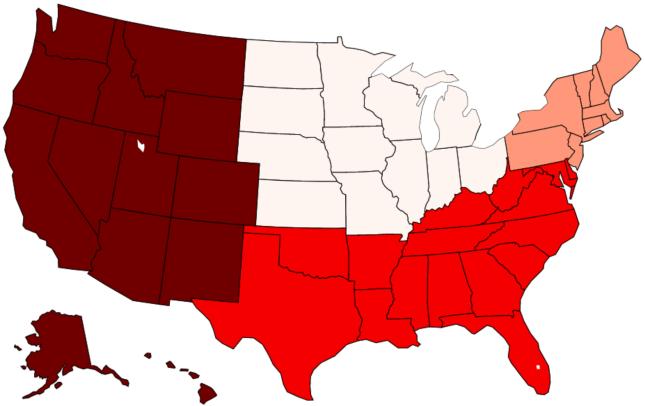
- `plotly`
- `sp`
- `sf`

# Maps in R

If you're feeling adventurous (or picky), we can use R's strength.
One can modify any image as you want. Thus, starting with a map and modifying it is an option. It is rarely easy, but it is possible, and may be well worth it.
You can even create maps from scratch, adding several features to them to increase the data density.

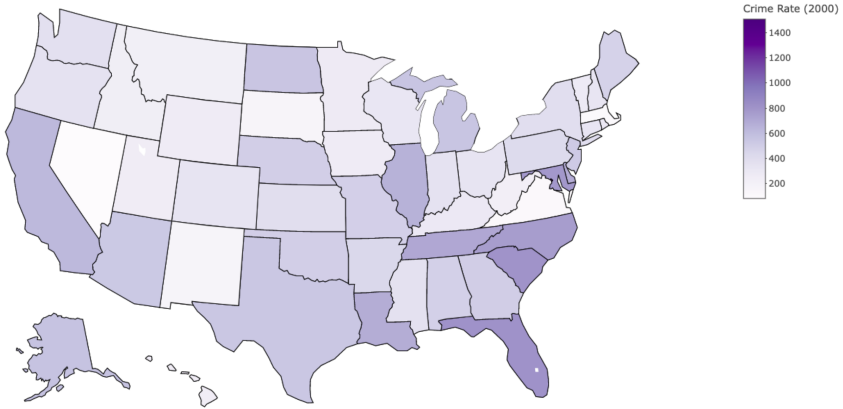
# Plotly Map of Categorical Data

Here is a map of the four census regions using `plotly`:



# Plotly Map of Numeric Data

Here is a map of the violent crime rate in 2000 using `plotly`:



# A Map from Scratch

Here is a map Dr. Forsberg made from scratch. I'm not sure how he did this, I'll have to bug him!

